Sentiora

Technical Specification v1.2

Final Engineering Execution Handbook

Architecture → Implementation → Testing → Deployment

Status: FINAL FOR MVP IMPLEMENTATION **Date:** July 2026 **Prepared for:** Sentiora Four-Person Engineering Team
Supersedes: Technical Specification v1.1 (readability/structure only — architecture unchanged)

Table of Contents

1. Purpose & How To Use This Document

This handbook turns the approved Sentiora v1.1 architecture into something you can keep open beside VS Code while building. It does not change any architecture decision — it makes the same decisions easier to read, visualize, and follow phase by phase.

How to use it: - Find your phase in Section 8 → read Goal, Owner, Tasks, DoD, Tests → start coding. - Before writing any endpoint, check the API Implementation Checklist (Section 11). - Before merging, run the Before You Merge Checklist (Section 15). - If you hit a `Permission`, sensitive-content, isolation, or deletion question, check the relevant Security Gate (Section 12).

> **Nothing in this document changes v1.1's architecture.** Where anything here could be read as a new decision, v1.1 wins.

---

2. Preserved Architecture — What Is Locked In

The following are approved and **not open for reconsideration** during MVP build:

Layer	Locked Decision
Chrome Extension	Manifest V3, TypeScript, React
Web App	React + Vite, TypeScript
Backend	FastAPI (Python), modular monolith
Primary Database	PostgreSQL
Vector Storage	pgvector (inside PostgreSQL — **not** ChromaDB/Qdrant)
AI/RAG Orchestration	Direct provider API calls — **not** LangChain/LlamaIndex
Embedding	Provider-hosted embedding API
LLM	Single provider-hosted LLM API
Authentication	JWT access token + Redis-backed rotating refresh token, argon2 hashing
Background Processing	Redis + RQ workers
Object Storage	S3-compatible, where applicable
Deployment	Docker Compose, single managed container host — **not** Kubernetes
Repository	Monorepo
Testing	pytest, Vitest + React Testing Library, Playwright
CI/CD	GitHub Actions
Isolation	Strict `user_id` scoping on every query
Privacy	Permission-before-collection, layered sensitive-content exclusion
Capture	Meaningful Capture Engine, asynchronous ingestion
Search/AI	Semantic + keyword search, RAG with citation validation
Deletion	Hard delete of Memory + MemoryChunks + embeddings

Explicitly future, not MVP: Sentiora Footprint, Sentiora Shield. They may be referenced as extension points only — never implemented here.

Explicitly rejected for this MVP: microservices, Kubernetes, Kafka/event streaming, event sourcing, service mesh, complex observability platforms, multiple vector databases, premature distributed architecture. Four developers build one deployable, understandable system.

---

3. Technology Stack Reference

Layer	Selected Technology	Why	Main Trade-off
Chrome Extension	Manifest V3, TypeScript, React	MV3 mandatory for the Web Store; TypeScript safety across 4 developers	Non-persistent service worker adds lifecycle complexity
Web Frontend	React + Vite, TypeScript	Fast dev loop; shares components with the extension	Needs an auth bridge between extension and web app
Backend	FastAPI (Python 3.12)	Async-first, fastest path to a RAG-heavy API for a small team	Python concurrency ceiling at very high scale (not an MVP concern)
Primary Database	PostgreSQL	Relational integrity for ownership chains; supports pgvector	—
Vector Storage	pgvector	One database, atomic delete cascades, adequate MVP performance	—
AI/RAG Orchestration	Direct provider API calls	Full control of citations and prompt-injection defenses	More boilerplate than a framework
Embedding Model	Provider-hosted API	Avoids hosting a model; low cost	Vendor dependency
LLM Strategy	Single provider-hosted API	Fastest path to grounded RAG chat	Vendor dependency; outage triggers fallback behavior
Authentication	JWT + Redis-backed refresh token, argon2	Stateless verification, revocable sessions	Requires a refresh-token store
Background Processing	RQ + Redis	Smallest operational footprint	Redis becomes required infra (also used for rate limiting)
Object Storage	Local disk (dev) / S3-compatible (staging/prod)	Keeps Postgres rows small	One more managed dependency in prod
Deployment	Docker Compose + single managed host	Avoids Kubernetes; identical images everywhere	Manual scaling, acceptable at MVP scale
Testing	pytest, Vitest + RTL, Playwright	Mature, low-friction, matches stack	—
CI/CD	GitHub Actions	Free, integrates with the monorepo	—

---

4. How Sentiora Works — End-to-End

Scenario: a user spends four minutes reading a Docker article, then later asks Sentiora about it.

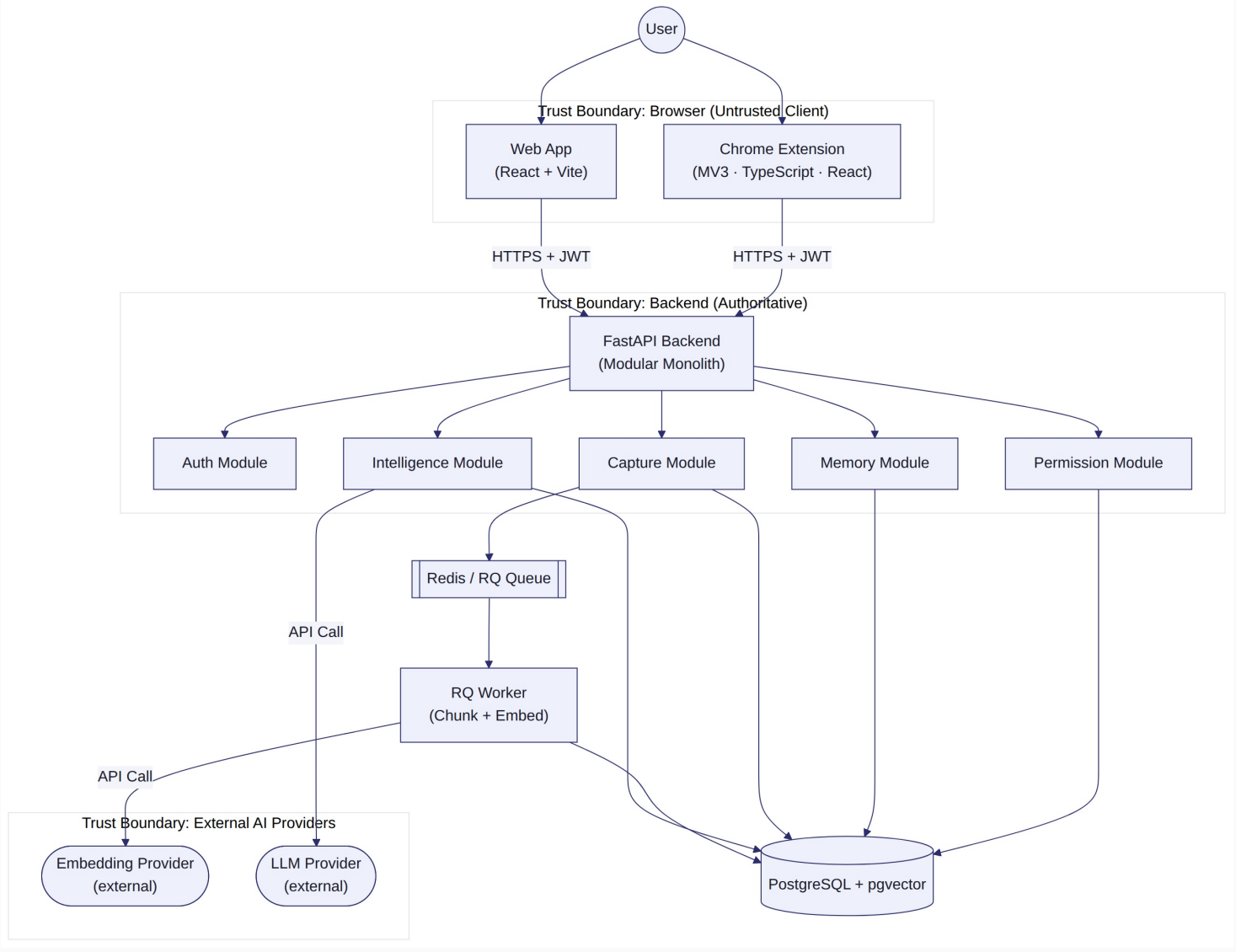
Step	What Happens
1. Permission	The extension's content script checks the locally cached `Permission` state for `webpage` capture. If off, nothing further happens.
2. Engagement	If on, dwell time, scroll depth, and revisit count are measured while the user reads.
3. Meaningful Capture	The Meaningful Capture Engine scores engagement against configured thresholds to decide if the page is worth remembering.
4. Privacy Guard	Client-side sensitive-content heuristics run before extraction — login, payment, Incognito contexts are excluded first.
5. Extraction	If both checks clear, the content script extracts title, URL, and visible text.
6. Backend validation	The extension POSTs to `/api/v1/capture`. FastAPI verifies the JWT, resolves `user_id` server-side, and **re-runs** permission and sensitive-content checks — the client is never trusted alone.
7. Memory creation	A `Memory` row is created synchronously with `processing_status = pending`.
8. Async processing	An RQ job normalizes, deduplicates, and chunks the content.
9. Vector storage	Each chunk is embedded and stored with its `MemoryChunk` row; `processing_status` becomes `ready`.
10. Recall	Later the user asks "*Where did I learn about Docker containers?*" Sentiora embeds the query, runs a `user_id`-scoped pgvector search, retrieves the article's chunks, builds a delimited context, calls the LLM, validates citation tags, and

	returns a grounded, cited answer.

5. Visual Architecture

This section contains the ten required system diagrams. Each is followed by a short explanation.

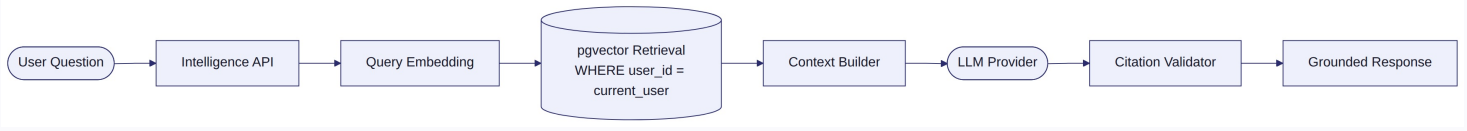
5.1 Diagram 1 — Sentiora High-Level System Architecture



Full request/data path from User through the Chrome Extension and Web App, into the FastAPI modular monolith, through Redis/RQ async processing, into PostgreSQL + pgvector, out to external embedding/LLM providers. Shaded regions mark the three trust boundaries: the browser (untrusted client

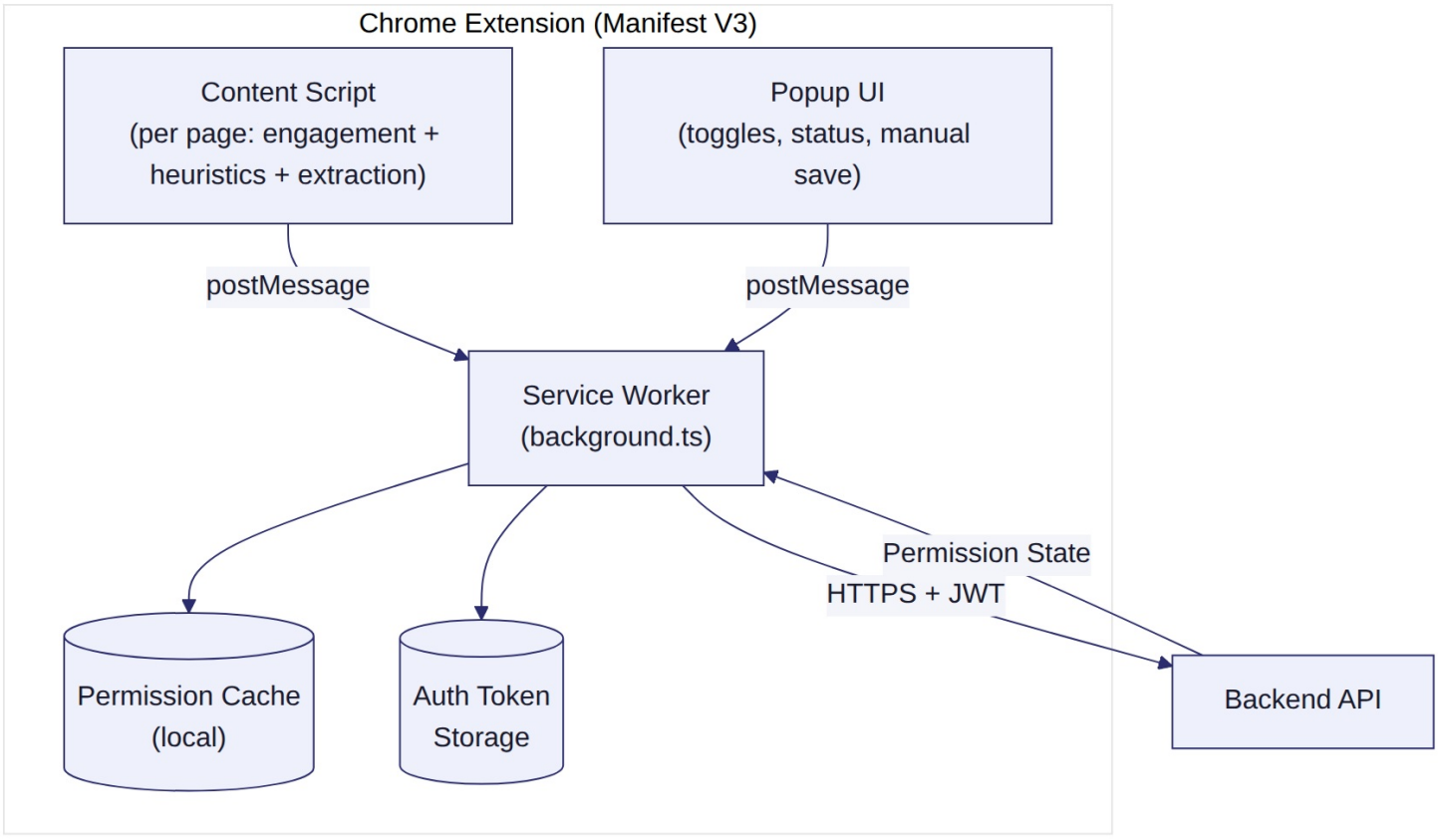
, the backend (authoritative), and external AI providers (outbound-only, no credentials leave the server).)

5.2 Diagram 1b — Ask Sentiora Request Path (Summary View)



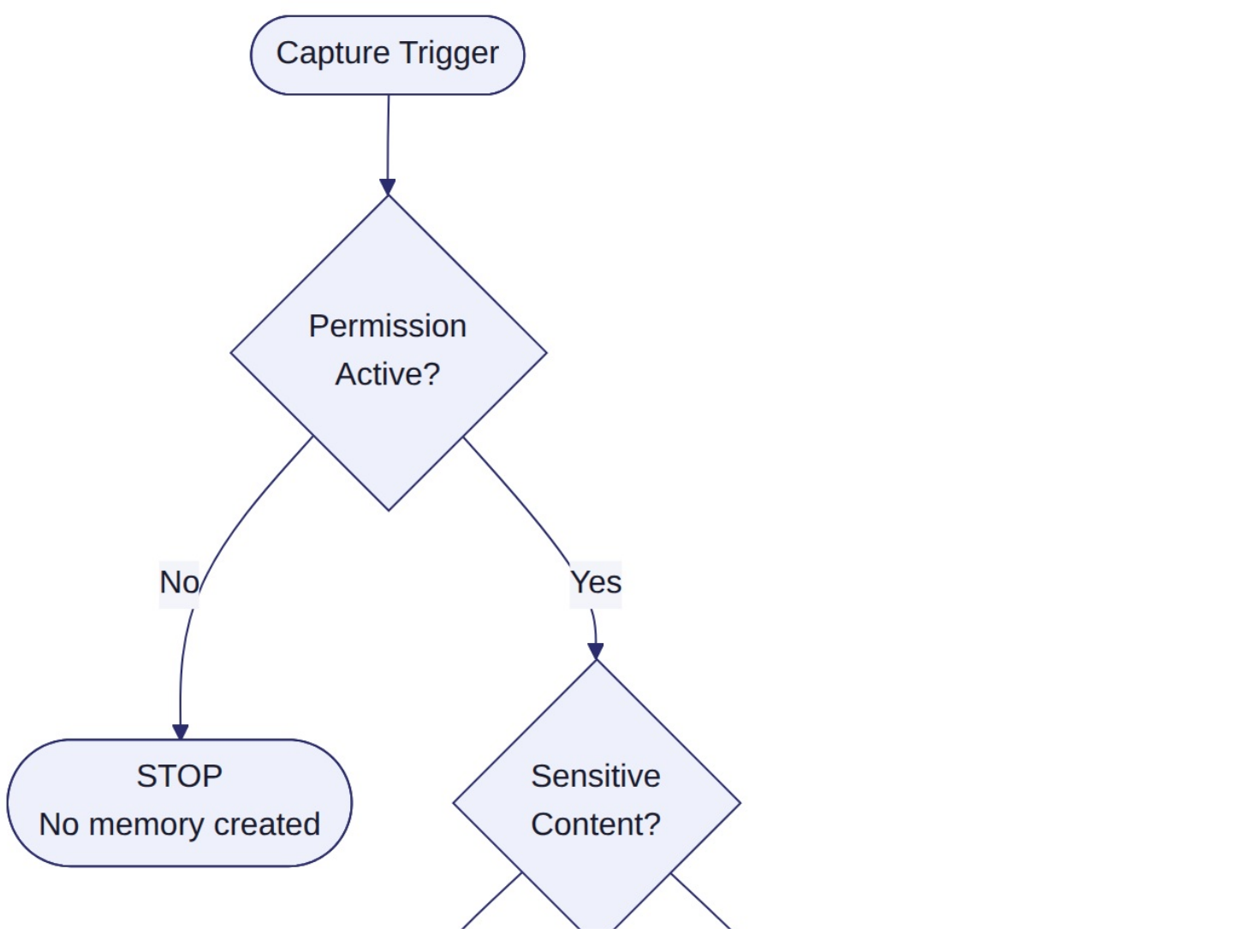
A condensed view of the RAG path referenced from the HLD: Question → Intelligence API → Query Embedding → pgvector Retrieval → Context Builder → LLM → Citation Validator → Grounded Response. See Diagram 7 for the full branching version with the insufficient-evidence path.

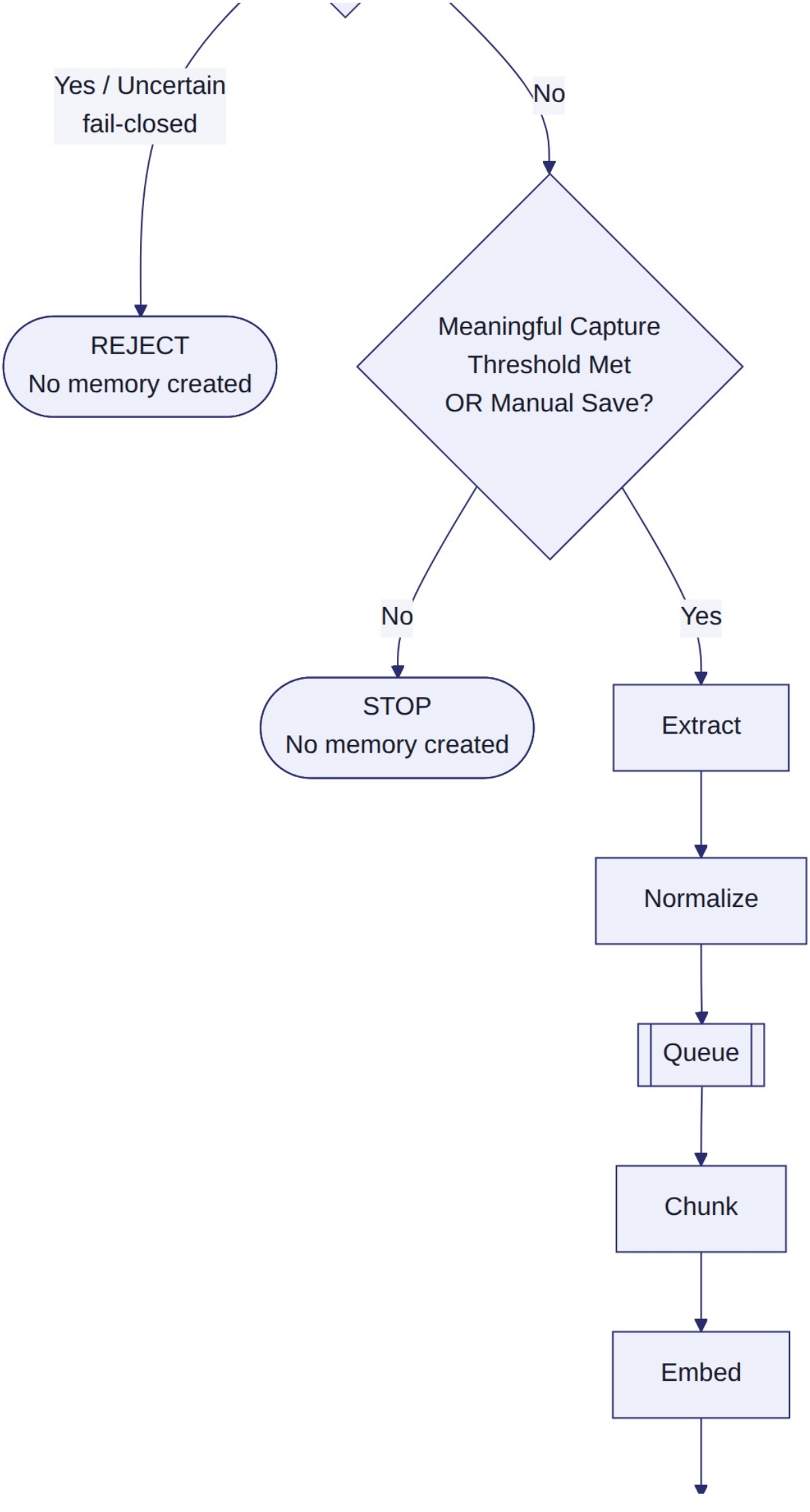
5.3 Diagram 2 — Chrome Extension Internal Architecture



Internal message flow inside the extension: Content Script and Popup UI both talk only to the Service Worker via typed messages; the Service Worker owns the Permission Cache and Auth Token storage and is the only piece that calls the Backend API.

5.4 Diagram 3 — Permission + Privacy Capture Decision Flow

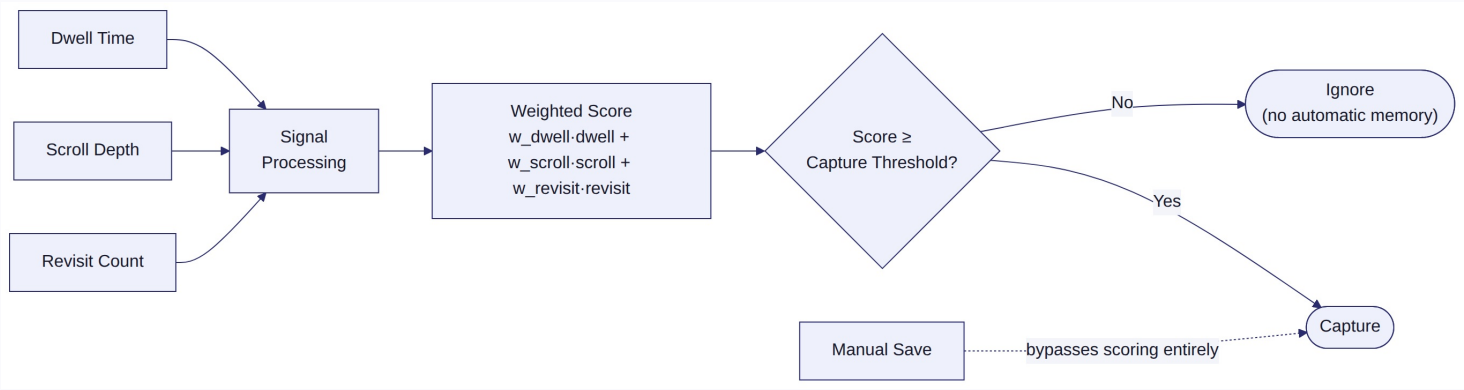



Yes / Uncertain
fail-closed
No
REJECT
No memory created
Meaningful Capture
Threshold Met
OR Manual Save?
No
Yes
STOP
No memory created
Extract
Normalize
Queue
Chunk
Embed

Every capture attempt passes through two decision diamonds — Permission Active? and Sensitive Content? — before the Meaningful Capture Engine even runs. Any "No" or "Uncertain" branch on the sensitive-content check stops the flow with no memory created (fail-closed

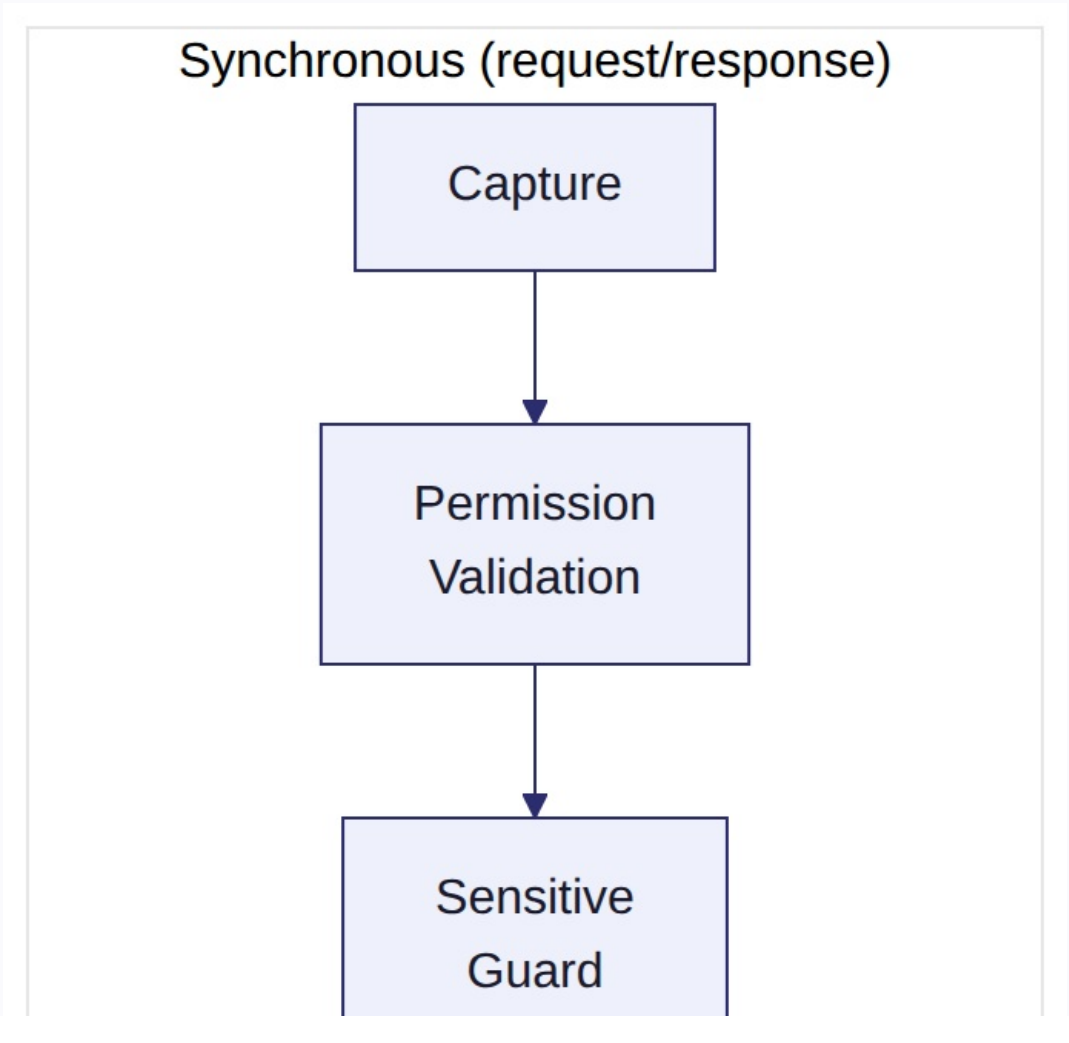
.)

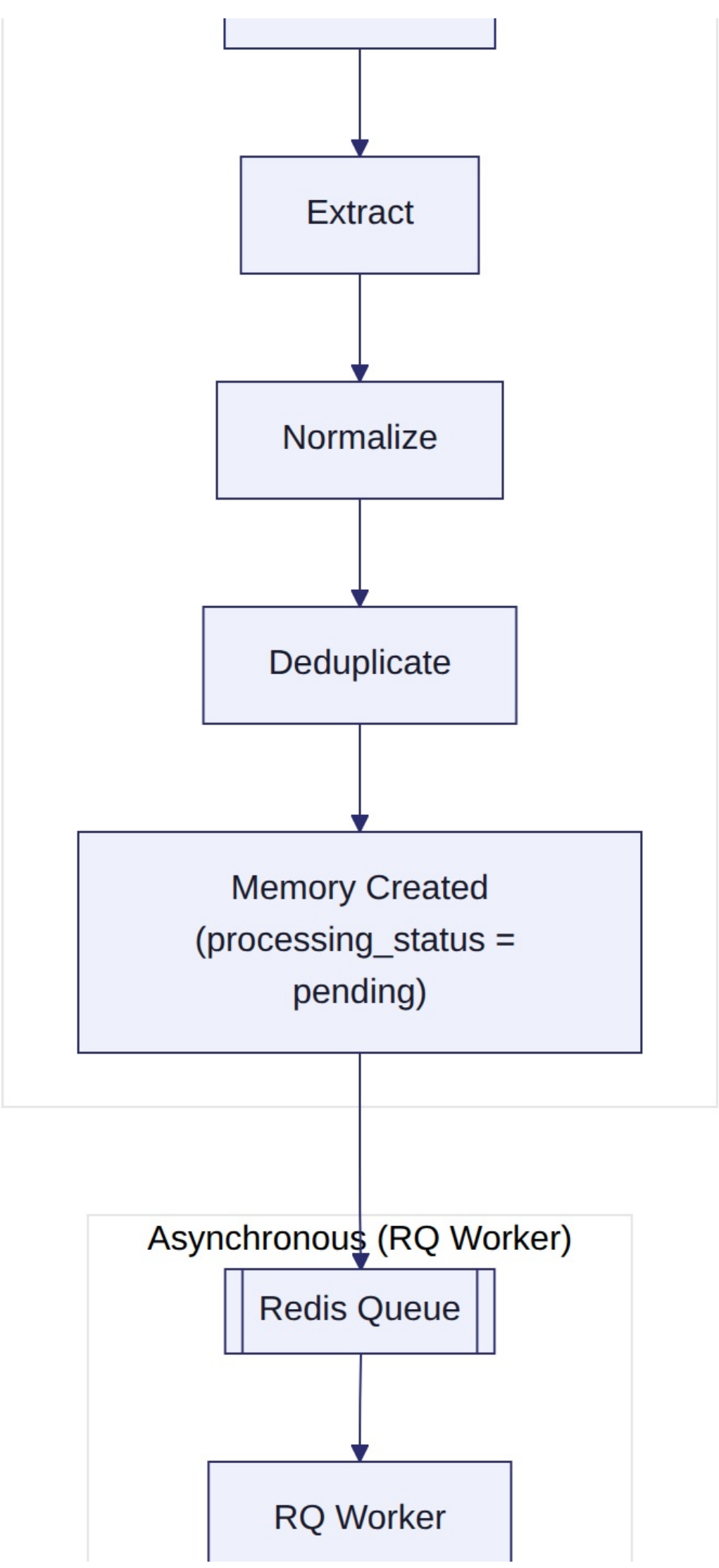
5.5 Diagram 4 — Meaningful Capture Engine Flow

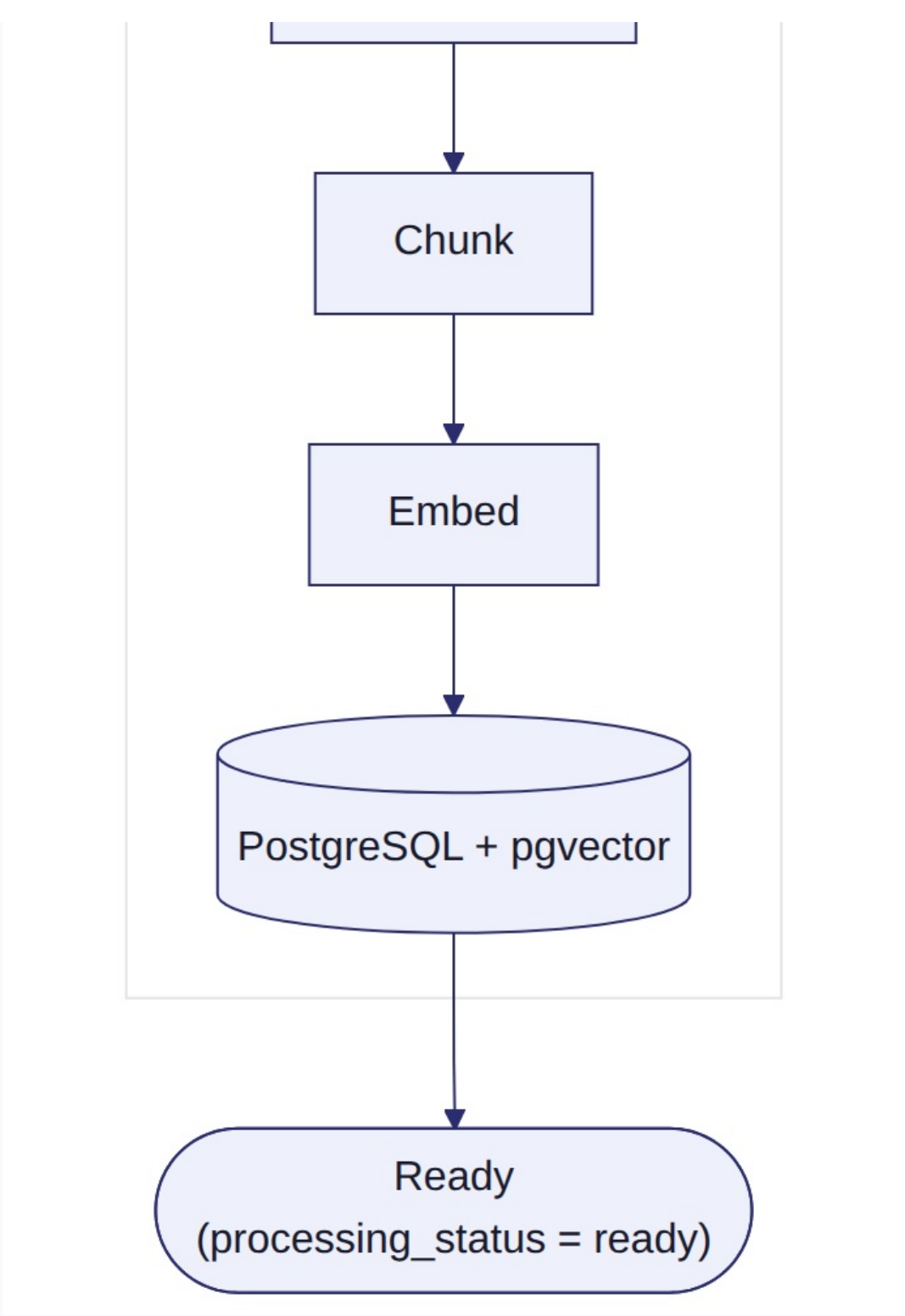


Dwell time, scroll depth, and revisit count feed a weighted score, compared against a capture threshold. Manual Save is shown as a dashed line bypassing scoring entirely — it always succeeds regardless of the automatic-capture decision.

5.6 Diagram 5 — Ingestion Pipeline (Sync vs. Async Boundary)

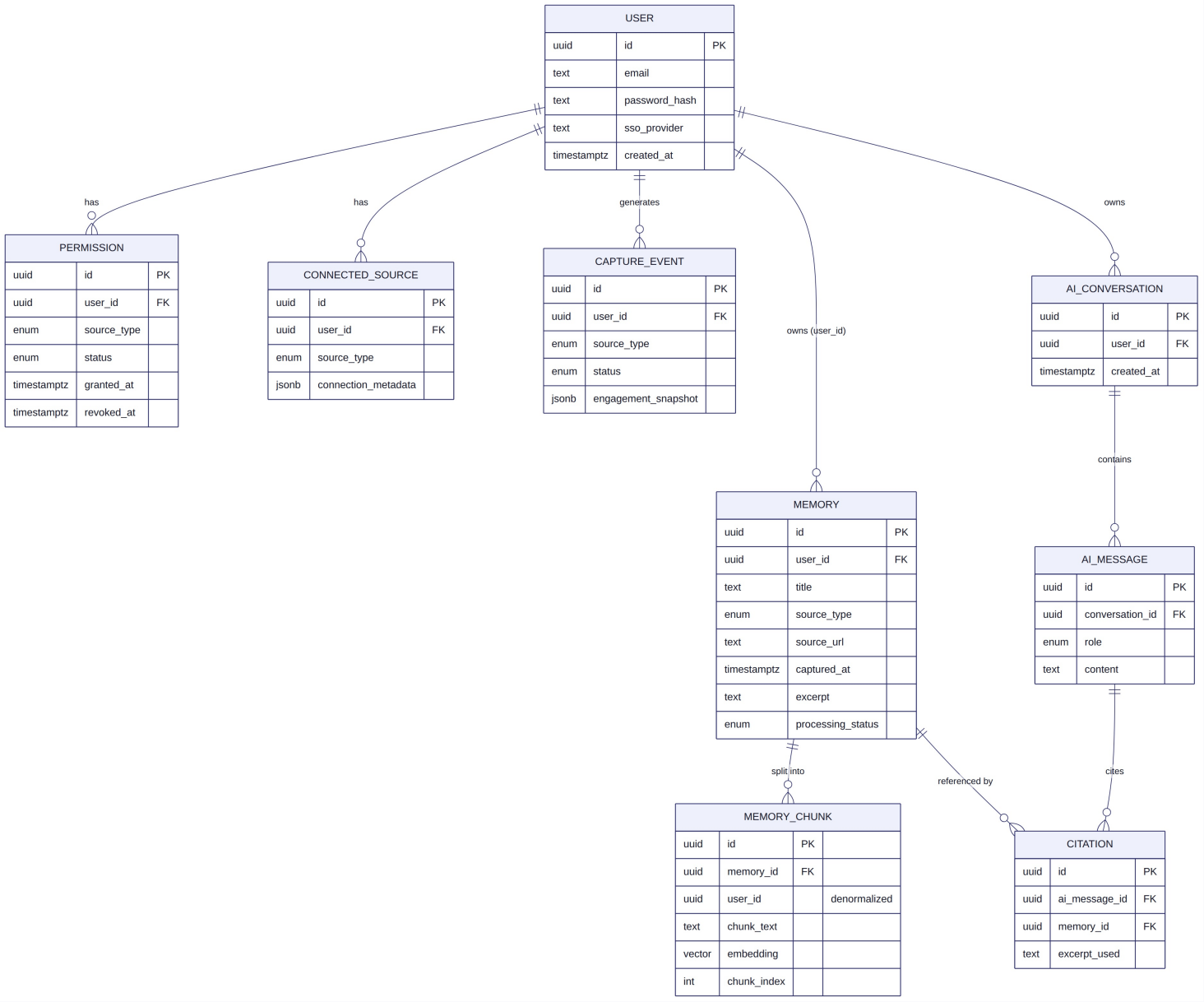



Extract
Normalize
Deduplicate
Memory Created (processing_status = pending)
Asynchronous (RQ Worker)
Redis Queue
RQ Worker



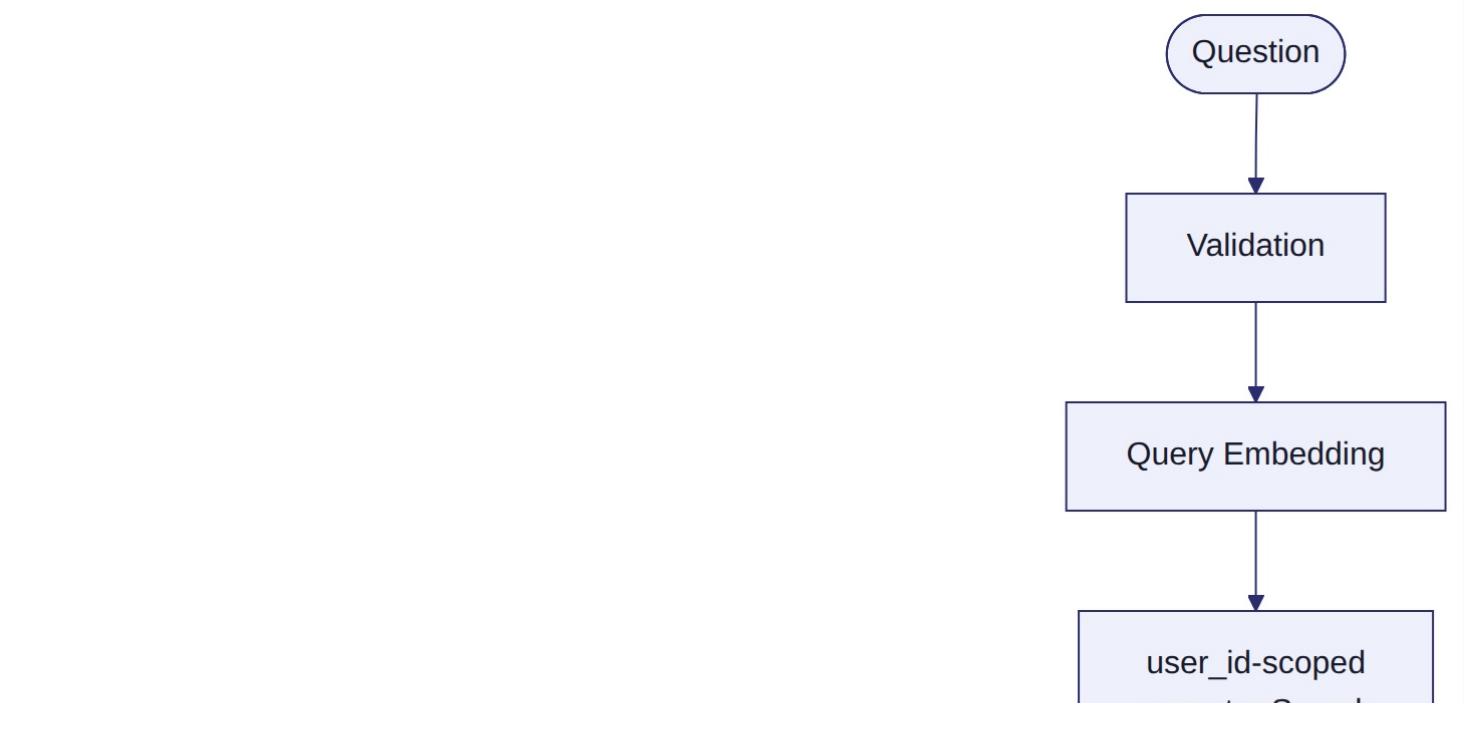
The shaded synchronous region — Capture through Memory Created — completes within the original HTTP request. Everything past the Redis Queue boundary runs asynchronously on an RQ Worker, so browsing is never blocked while chunking and embedding happen.

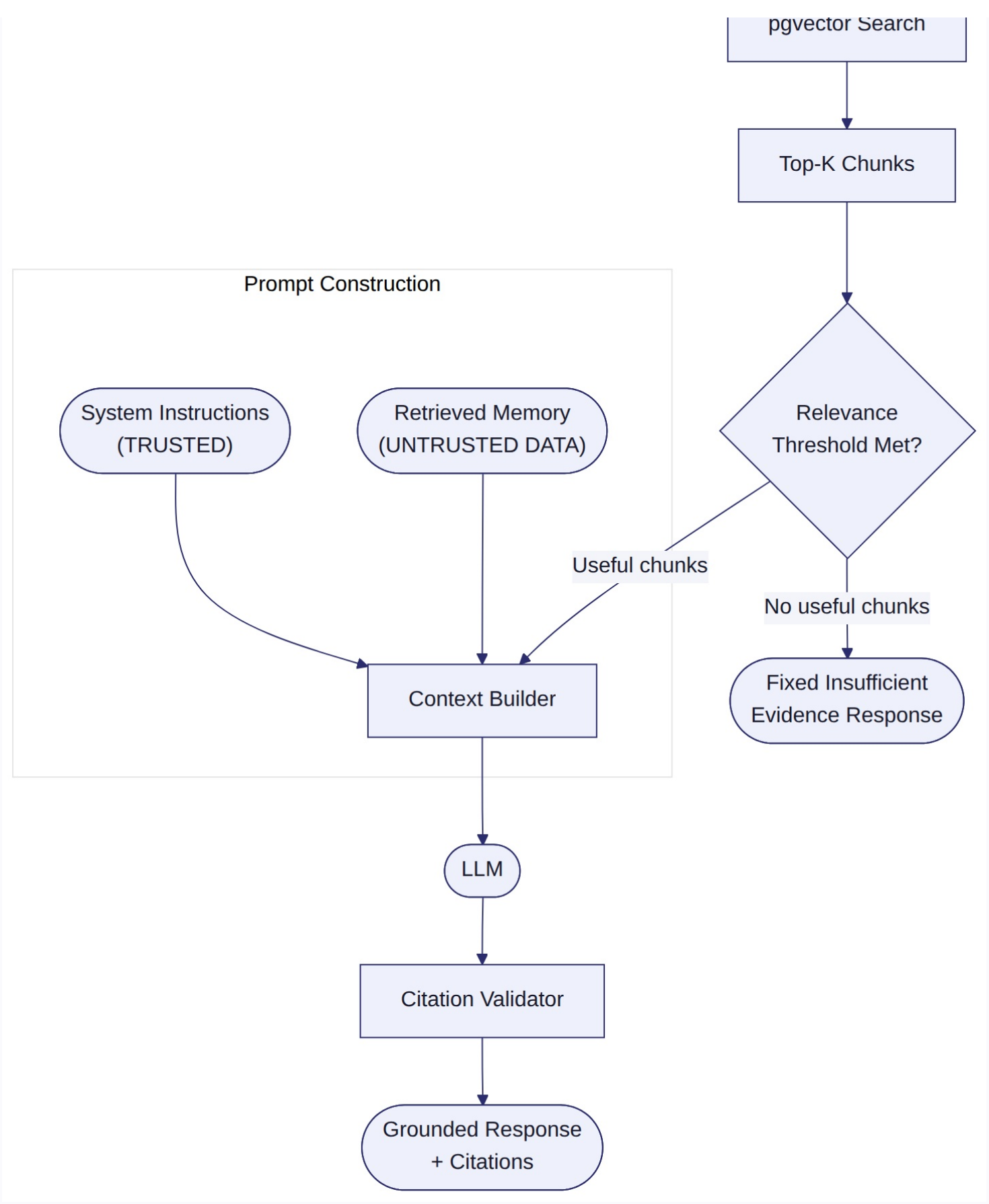
5.7 Diagram 6 — Database ERD



Every owned entity carries a user_id foreign key back to User, making cross-user isolation visually explicit at the schema level. MemoryChunk carries the pgvector embedding column and a denormalized user_id for isolation-safe vector queries. Citation is the only entity referencing Memory from outside the direct ownership chain, and is validated at query time to ensure it shares the same user_id as its parent AIMessage.

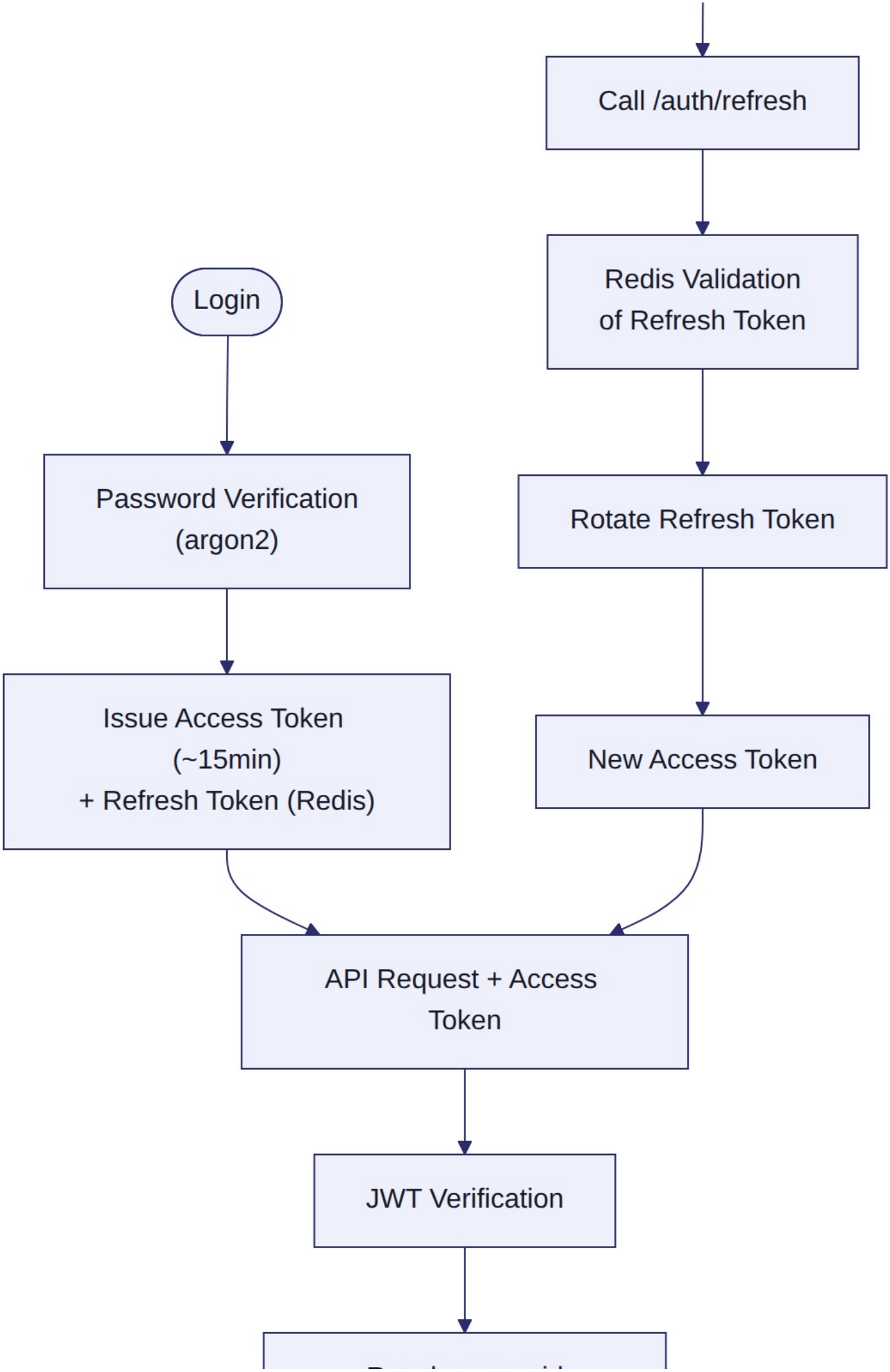
5.8 Diagram 7 — RAG / Ask Sentiora Pipeline (Full)

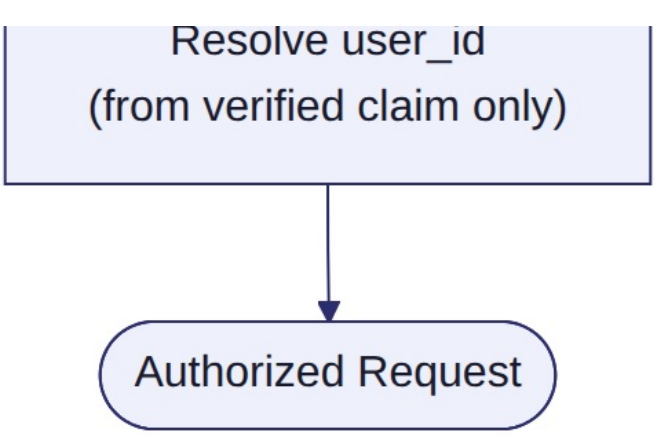




The complete branching RAG flow, including the insufficient-evidence short-circuit when no chunk clears the relevance threshold. The Prompt Construction box makes the trust boundary explicit: System Instructions are trusted; Retrieved Memory is always treated as untrusted data, never as instructions.

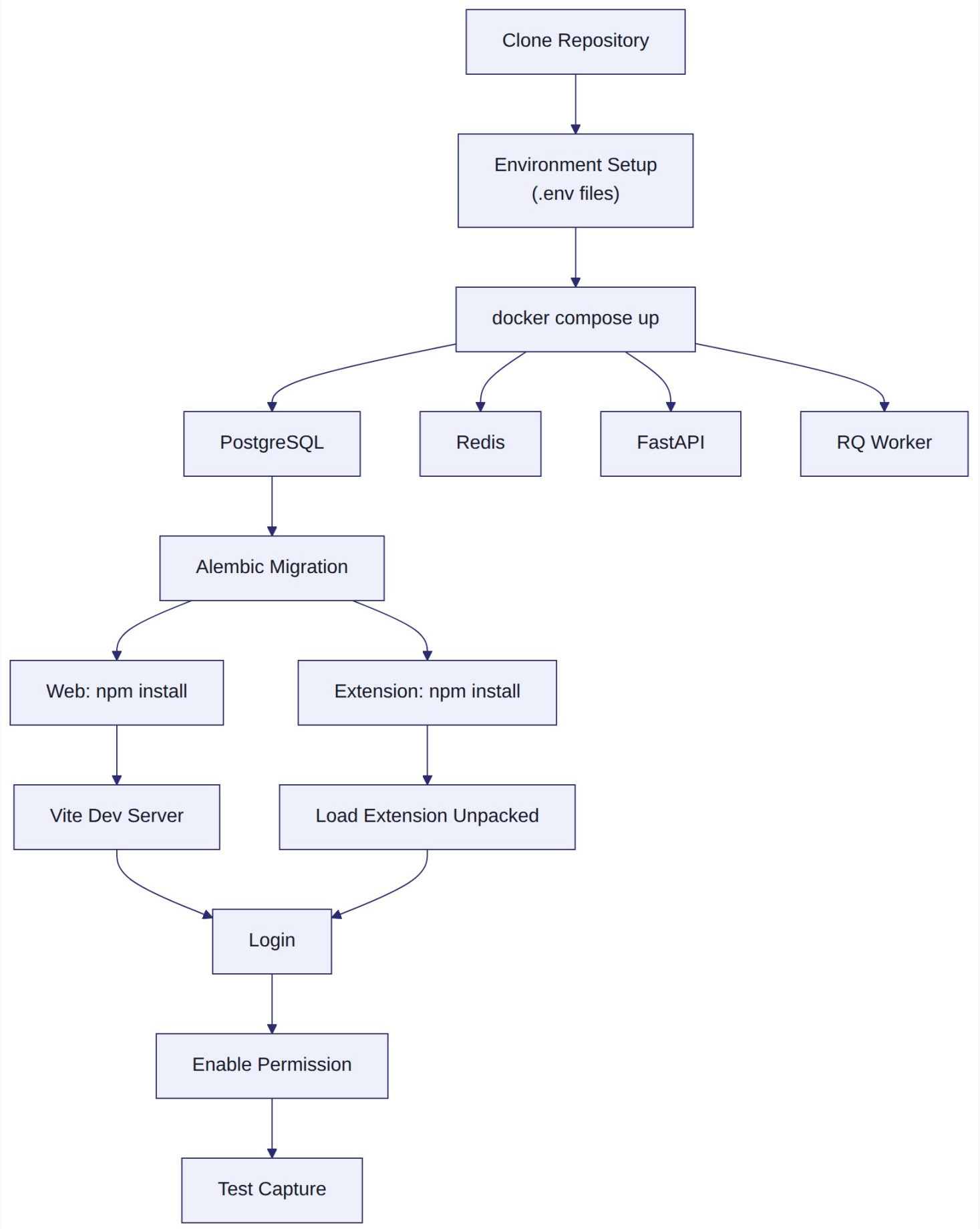
5.9 Diagram 8 — Authentication Flow


Call /auth/refresh
Redis Validation of Refresh Token
Login
Rotate Refresh Token
Password Verification (argon2)
Issue Access Token (~15min) + Refresh Token (Redis)
New Access Token
API Request + Access Token
JWT Verification



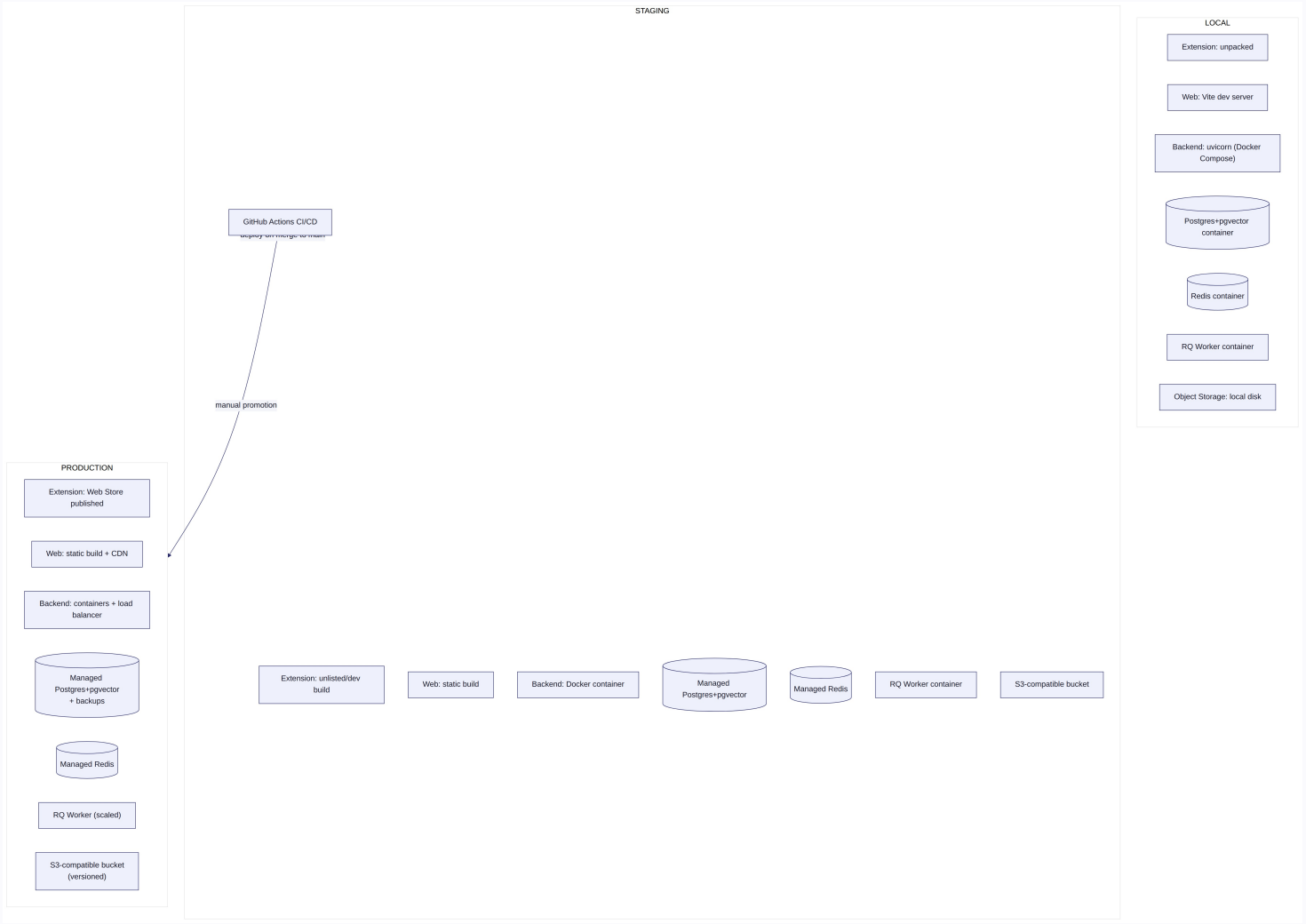
Top path: login through password verification to an authorized request. Bottom path: what happens when the access token expires — refresh, Redis validation, rotation, and a new access token, looping back into the same authorized-request path.

5.10 Diagram 9 — Local Development Flow



The exact sequence a developer follows from a fresh clone to a working local capture test. See Section 9 for the same flow narrated step-by-step with commands.

5.11 Diagram 10 — Deployment Architecture

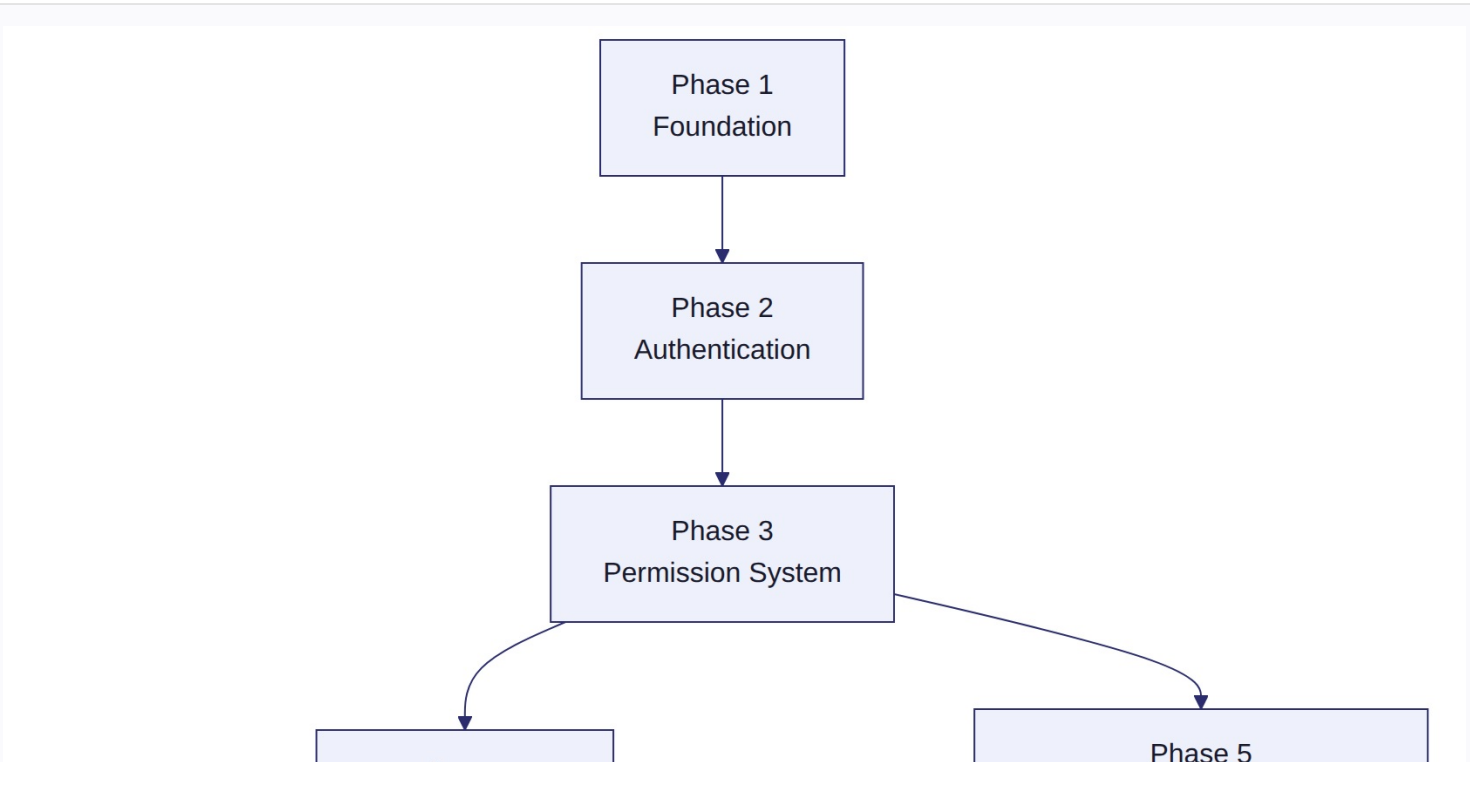


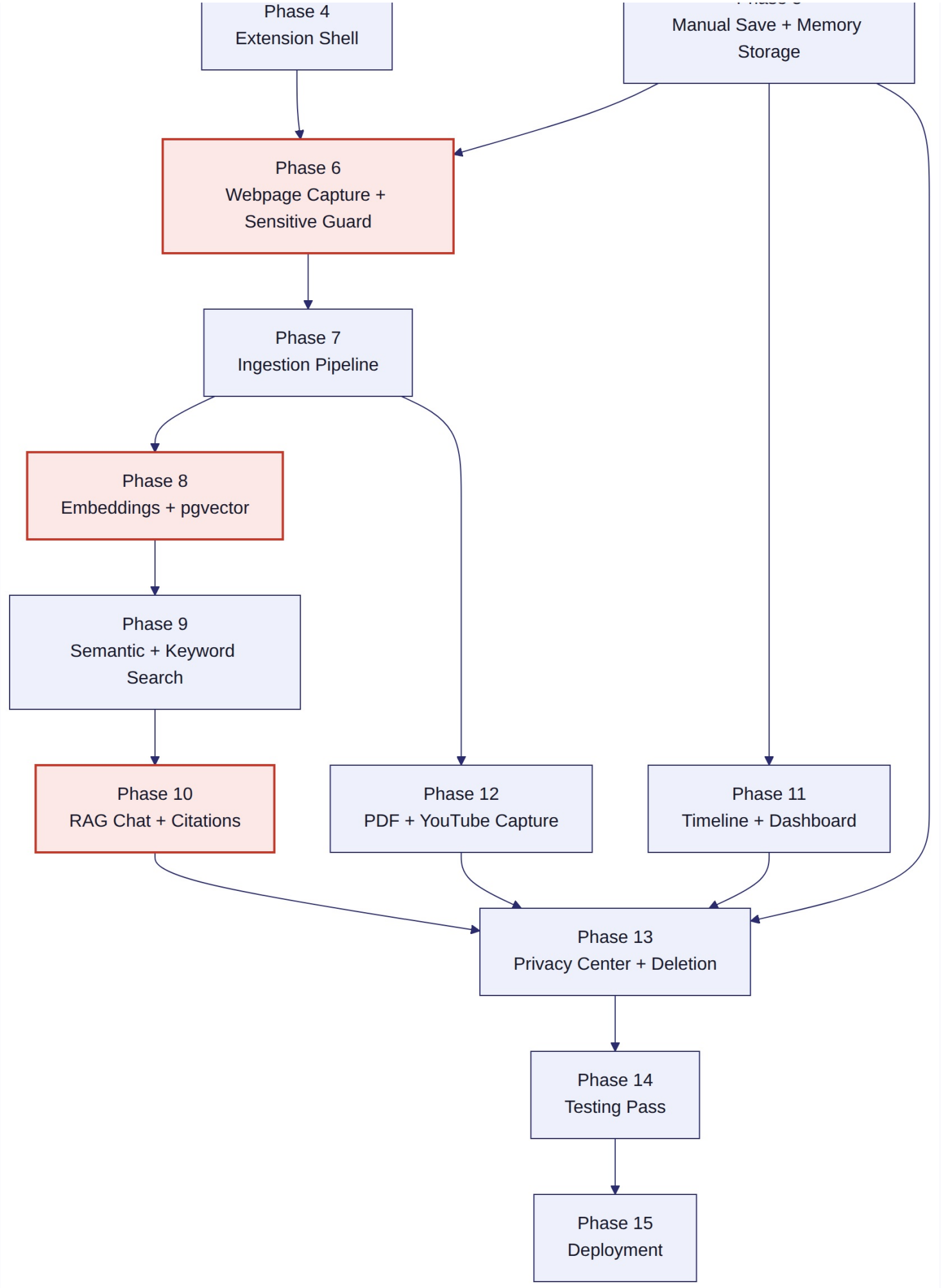
Local, Staging, and Production shown side by side with every component present in each. GitHub Actions CI/CD deploys to Staging automatically on merge to main, and to Production via manual promotion.

---

6. Master Implementation Roadmap

This single view answers: what must happen first, what's blocked, what can run in parallel, and what depends on what. Phases outlined in red are gated by a blocking TBD (Section 13).





All 15 v1.1 build phases with their dependency edges. Phase 3 (Permissions

is the fork point: Phase 4 (Extension Shell) and Phase 5 (Manual Save + Memory Storage) can proceed in parallel once it lands. Phase 6 (Webpage Capture) needs both and is the first red-flagged phase — blocked on TBD-1. Phase 8 (Embeddings) and Phase

10 (RAG Chat) are blocked on TBD-2. Phases 11 and 13 (Timeline/Dashboard, Privacy Center) can start once Memory APIs (Phase 5) stabilize, without waiting for search or RAG.)

Reading the roadmap: - **Must happen first:** Phase 1 → Phase 2 → Phase 3. Nothing else can start before Permissions exist. - **Can run simultaneously:** Extension Shell (4) and Manual Save/Memory Storage (5) once Phase 3 lands; Timeline/Dashboard (11) once Phase 5 lands, independent of search/RAG progress. - **What AI depends on:** Phase 10 (RAG) depends on Phase 9 (Search) which depends on Phase 8 (Embeddings) which depends on Phase 7 (Ingestion). - **What the extension depends on:** Phase 4 depends only on Auth (2) and Permissions (3); it does not wait for the backend's capture logic to be complete to begin its own shell work. - **What frontend depends on:** the Web App's Timeline/Dashboard (11) only needs Memory APIs (5) to be stable — it should not wait for Phase 10.

---

7. Four-Person Parallel Development Roadmap

Roles:

Role	Owns
Team Member 1 — Project Lead	Architecture, integration, Database + RAG
Team Member 2 — Extension	Chrome Extension
Team Member 3 — Backend	Backend / API
Team Member 4 — Web Frontend	Web App

Security is shared across all four. Testing is owned by the feature developer and reviewed by a different developer.

Swimlane — Phase Progression by Owner

Phase	Project Lead	Extension	Backend	Web Frontend	Coordination Note
1. Foundation	Repo scaffold, CI skeleton	—	Docker Compose services	—	Lead sets up the monorepo before anyone else starts
2. Authentication	Reviews contracts	—	Builds `auth/` module	Login/register forms (once contracts exist)	Backend defines the token contract first; Web waits on it
3. Permissions	Reviews contracts	—	Builds `permissions/` module	Connected Sources screen	Backend and Web can pair once the endpoint shape is agreed
4. Extension Shell	—	MV3 skeleton, popup, token storage	—	—	Runs in parallel with Phase 5
5. Manual Save + Memory	Schema review	—	`Memory` model + manual endpoint	Manual note UI	Runs in parallel with Phase 4
6. Webpage Capture + Sensitive Guard	**Pairs on rules**	**Content script extraction + client heuristic**	**Server-side re-check + MCE scoring**	—	**Critical integration point — Extension + Backend pair directly, do not work in isolation**
7. Ingestion Pipeline	Reviews dedup/chunk logic	—	RQ jobs, chunking, dedup	—	Backend-owned, Lead reviews
8. Embeddings + pgvector	**Leads this phase**	—	Provider integration support	—	Project Lead + Backend jointly own this
9. Semantic + Keyword Search	**Leads this phase**	—	Search endpoint support	Search bar UI	Lead + Backend own retrieval logic; Web consumes the endpoint
10. RAG Chat + Citations	**Leads this phase**	—	Context builder, citation validator support	Chat UI	Project Lead + Backend pair directly
11. Timeline + Dashboard	—	—	Memory list endpoint	Timeline, Dashboard	Can start once Phase 5 lands, independent of Phases 8–10

			support	screens	
12. PDF + YouTube Capture	—	Extraction logic	Source-type handling	—	Extension + Backend, same pattern as Phase 6
13. Privacy Center + Deletion	Reviews cascade logic	—	Delete cascade hardening	Privacy Center screen	Backend + Web, Lead reviews deletion correctness
14. Testing Pass	Coordinates suite	Extension E2E	Backend + isolation suite	Web E2E	All four contribute; Lead coordinates coverage
15. Deployment	Owns staging/prod	—	Container readiness	Static build readiness	Lead-owned with support from Backend

Where pair-programming is recommended: Phase 6 (Webpage Capture — Extension + Backend), Phase 8–10 (Embeddings/Search/RAG — Project Lead + Backend). **Where cross-review happens:** every phase's tests are reviewed by a developer other than the one who wrote the feature, per the shared-testing rule above.

---

8. The 15-Phase Build Guide

Every phase below uses the same structure. If a category doesn't apply to a phase, it reads **N/A**.

PHASE 1 — Foundation

Goal: A working, empty full-stack skeleton the whole team can build on. **Why this phase exists:** Nothing else can start without a shared repo, local environment, and CI baseline. **Dependencies:** None. **Primary Owner:** Project Lead. **Reviewer:** Team (all four review the initial structure). **Modules / folders affected:** `infrastructure/`, `docker-compose.yml`, `.github/workflows/`, root `.gitignore`, `.env.example` files. **Backend Tasks:** create `backend/app/` skeleton with a `/health` route. **Frontend Tasks:** N/A (Web App scaffold happens in Phase 5/11 setup, not here). **Extension Tasks:** N/A (extension scaffold happens in Phase 4). **Database Tasks:** N/A — no schema yet. **Security Requirements:** `.env.example` committed, real `.env` gitignored from day one. **Implementation Steps:** scaffold monorepo per Section 16 of v1.1 → write `docker-compose.yml` for Postgres/Redis/backend/worker → add a minimal FastAPI app with `/health` → add GitHub Actions lint+test job. **API Endpoints involved:** `GET /health`. **Expected Inputs / Outputs:** none in, `{"status": "ok"}` out. **Definition of Done:** `docker compose up` brings up an empty FastAPI app responding on `/health`. **Required Tests:** CI pipeline runs and passes on an empty app. **Common Failure Cases:** missing `.env.example` causing a teammate's Compose stack to fail silently; port collisions between Postgres and a local install. **Git Branch Example:** `feature/repo-foundation` **Suggested PR Title:** `chore: scaffold monorepo, Docker Compose, and CI skeleton`

PHASE 2 — Authentication

Goal: Users can register, log in, and hold a session. **Why this phase exists:** Every other phase requires a resolvable, trustworthy `user_id`. **Dependencies:** Phase 1. **Primary Owner:** Backend. **Reviewer:** Project Lead. **Modules / folders affected:** `backend/app/auth/`, `web/src/pages/` (login/register). **Backend Tasks:** register, login, refresh, logout endpoints; argon2 hashing; JWT issuance; Redis-backed refresh-token store. **Frontend Tasks:** login/register forms, token storage, auth-aware routing. **Extension Tasks:** N/A (extension login happens in Phase 4). **Database Tasks:** `user` table, Alembic migration. **Security Requirements:** passwords hashed with argon2, never logged; JWTs signed with a secret from environment config, never hardcoded; refresh tokens revocable via Redis. **Implementation Steps:** define `User` model → migration → implement register/login/refresh/logout → wire Web App forms against the new endpoints. **API Endpoints involved:** `POST /api/v1/auth/register`, `POST /api/v1/auth/login`, `POST /api/v1/auth/refresh`, `POST /api/v1/auth/logout`. **Expected Inputs / Outputs:** see Section 13 API contracts (`/auth/login` example). **Definition of Done:** a new account can be created, logged into, and produces a valid access token accepted by a protected test route. **Required Tests:** invalid credentials rejected with a generic error; expired token rejected; refresh rotates correctly; logout invalidates the refresh token. **Common Failure Cases:** leaking which field (email vs. password) was wrong on login failure; forgetting to invalidate the refresh token on logout. **Git Branch Example:** `feature/auth-login` **Suggested PR Title:** `feat(auth): registration, login, refresh, and logout`

PHASE 3 — Permission System

Goal: Per-source consent is persisted and enforced. **Why this phase exists:** Permission-before-collection (Security Gate 2) cannot be enforced anywhere downstream without this. **Dependencies:** Phase 2. **Primary Owner:** Backend. **Reviewer:** Project Lead. **Modules / folders affected:** `backend/app/permissions/`, `web/src/pages/` (Connected Sources, onboarding). **Backend Tasks:** grant/revoke endpoints, onboarding-ack endpoint. **Frontend Tasks:** Connected Sources screen, onboarding gating (toggles disabled until privacy explanation acknowledged). **Extension Tasks:** N/A (extension reads permission state in Phase 4). **Database Tasks:** `permission` table, unique constraint on `(user_id, source_type)`, migration. **Security Requirements:** revocation takes effect immediately for all future requests; no permission can be created before onboarding acknowledgment.

Implementation Steps: define `Permission` model → migration → grant/revoke endpoints → onboarding-ack endpoint → Web App gating logic. **API Endpoints involved:** `GET /api/v1/permissions`, `POST /api/v1/permissions/{source_type}/grant`, `POST /api/v1/permissions/{source_type}/revoke`, `POST /api/v1/onboarding/ack`. **Expected Inputs / Outputs:** see Section 13 (`/permissions/{source_type}/grant` example). **Definition of Done:** toggling a source persists to the database and is checked server-side, not just reflected in the UI. **Required Tests:** disabling one source has no effect on others; revoke takes effect immediately; onboarding gate blocks toggles until acknowledged. **Common Failure Cases:** trusting a client-side toggle state without a server-side check; forgetting the unique constraint and allowing duplicate Permission rows per source. **Git Branch Example:** `feature/permissions` **Suggested PR Title:** `feat(permissions): per-source grant/revoke and onboarding gate`

PHASE 4 — Extension Shell

Goal: The extension can authenticate and read permission state. **Why this phase exists:** Establishes the extension's communication pattern before any capture logic is added. **Dependencies:** Phases 2, 3. **Primary Owner:** Extension. **Reviewer:** Project Lead. **Modules / folders affected:** `extension/src/background/`, `extension/src/popup/`, `extension/src/services/`. **Backend Tasks:** N/A (consumes existing Phase 2/3 endpoints). **Frontend Tasks:** N/A. **Extension Tasks:** MV3 skeleton, service worker, popup UI, token storage, `/permissions` fetch and local cache. **Database Tasks:** N/A. **Security Requirements:** auth token stored via extension-appropriate secure storage, never in `localStorage` accessible to page scripts; no provider API keys anywhere in extension code. **Implementation Steps:** scaffold MV3 manifest → build service worker message handling → build popup UI with toggle switches → wire login and permission-state fetch. **API Endpoints involved:** `POST /api/v1/auth/login`, `GET /api/v1/permissions`. **Expected Inputs / Outputs:** popup displays correct toggle states matching the backend's `Permission` rows. **Definition of Done:** loading the extension unpacked, logging in, and seeing correct toggle states in the popup. **Required Tests:** manual QA checklist now; automated E2E deferred to Phase 14. **Common Failure Cases:** requesting overly broad `host_permissions`; service worker losing state between wake cycles without re-fetching from the backend. **Git Branch Example:** `feature/extension-shell` **Suggested PR Title:** `feat(extension): MV3 shell, service worker, and popup`

PHASE 5 — Manual Save + Memory Storage

Goal: The first working Memory creation path. **Why this phase exists:** Establishes the `Memory` model and storage pattern that every capture source will reuse. **Dependencies:** Phase 3. **Primary Owner:** Backend. **Reviewer:** Project Lead. **Modules / folders affected:** `backend/app/memory/`, `backend/app/models/`, `web/src/pages/` (manual save UI). **Backend Tasks:** `Memory` model, migration, `capture/manual` endpoint. **Frontend Tasks:** manual note UI, submit flow, processing/success state. **Extension Tasks:** N/A. **Database Tasks:** `memory` table — user FK, source type, timestamps, `processing_status`. **Security Requirements:** `user_id` resolved from JWT only; manual save always succeeds for well-formed input. **Implementation Steps:** define `Memory` model → migration → `POST /capture/manual` → manual save UI. **API Endpoints involved:** `POST /api/v1/capture/manual`. **Expected Inputs / Outputs:** see Section 13 (`/capture/manual` example). **Definition of Done:** login → save note → row exists in the database → visible via a direct API call. **Required Tests:** User A creates and retrieves a memory; User B cannot retrieve User A's memory; invalid JWT rejected; empty note rejected. **Common Failure Cases:** forgetting `user_id` scoping on the retrieval query, allowing cross-user reads. **Git Branch Example:** `feature/manual-memory` **Suggested PR Title:** `feat(memory): Memory model and manual save endpoint`

PHASE 6 — Webpage Capture + Sensitive Content Guard

Goal: Meaningfully-engaged pages become memories; excluded pages never do. **Why this phase exists:** This is Sentiora's core trust mechanism — automatic capture must never violate the never-capture policy. **Dependencies:** Phases 4, 5. **Blocked by TBD-1 (health-content rule set) — see Section 13. Primary Owner:** Extension + Backend (paired — see Section 7). **Reviewer:** Project Lead. **Modules / folders affected:** `extension/src/content/`, `backend/app/capture/`, `backend/app/core/` (MCE constants). **Backend Tasks:** `capture/` webpage endpoint, server-side permission + sensitive-content re-check, Meaningful Capture Engine scoring (Section 8 formula in v1.1). **Frontend Tasks:** N/A. **Extension Tasks:** content script engagement tracking, client-side sensitive heuristic, extraction. **Database Tasks:** none new — writes to existing `memory` and `capture_event` tables. **Security Requirements: Security Gate 2 and Gate 3 both apply here** — server-side checks must never trust the client's judgment; fail-closed on any uncertain sensitive-content classification. **Implementation Steps:** implement client heuristic → implement matching server-side heuristic (must agree exactly) → implement MCE scoring per the documented formula → wire capture POST. **API Endpoints involved:** `POST /api/v1/capture`. **Expected Inputs / Outputs:** see Section 13 (`/capture` example). **Definition of Done:** a page held open past threshold becomes a Memory; a login page never does, even if manually revisited. **Required Tests:** threshold boundary cases; each sensitive category has a dedicated test page that must never produce a Memory. **Common Failure Cases:** client and server heuristics drifting out of sync; forgetting fail-closed behavior on ambiguous health-content classification. **Git Branch Example:** `feature/webpage-capture` **Suggested PR Title:** `feat(capture): webpage capture, sensitive-content guard, MCE scoring`

PHASE 7 — Ingestion Pipeline

Goal: Chunking, dedup, and async processing work end-to-end. **Why this phase exists:** Large or duplicate captures must not block browsing or corrupt the Memory store. **Dependencies:** Phase 6. **Primary Owner:** Backend. **Reviewer:** Project Lead. **Modules / folders affected:** `backend/app/workers/`. **Backend Tasks:** RQ chunk job, dedup logic (normalized-URL + content-

hash), RQ integration. **Frontend Tasks:** N/A. **Extension Tasks:** N/A. **Database Tasks:** none new — populates `memory_chunk` rows (unembedded at this stage). **Security Requirements:** idempotent processing via client-generated `capture_id`. **Implementation Steps:** implement normalize → dedup check → chunker → enqueue to Redis. **API Endpoints involved:** `GET /api/v1/capture/events/{id}` (status check). **Expected Inputs / Outputs:** a captured Memory produces correctly-sized `MemoryChunk` rows. **Definition of Done:** a large captured page is chunked correctly; a re-captured duplicate URL is deduplicated or linked, not silently duplicated. **Required Tests:** oversized document does not block the capture request; duplicate `capture_id` is a no-op. **Common Failure Cases:** chunking synchronously and blocking the HTTP response; silently dropping data on an uncertain dedup match instead of retaining both. **Git Branch Example:** `feature/ingestion` **Suggested PR Title:** `feat(ingestion): async chunking and deduplication via RQ`

PHASE 8 — Embeddings + pgvector Storage

Goal: Chunks are embedded and queryable. **Why this phase exists:** Nothing in Search or RAG works without embedded, indexed chunks. **Dependencies:** Phase 7. **Blocked by TBD-2 (embedding/LLM provider selection) — see Section 13. Primary Owner:** Project Lead + Backend. **Reviewer:** Backend / Project Lead (cross-review). **Modules / folders affected:** `backend/app/workers/`, `backend/app/models/` (`MemoryChunk.embedding`). **Backend Tasks:** embed RQ job, provider integration, `MemoryChunk.embedding` population. **Frontend Tasks:** N/A. **Extension Tasks:** N/A. **Database Tasks:** enable the `pgvector` extension; HNSW index on `MemoryChunk.embedding`. **Security Requirements:** only already-approved Memory content is sent to the embedding provider (never unapproved or excluded content). **Implementation Steps:** enable extension → add vector column + index → implement embed job with retry/backoff → populate embeddings for chunked content. **API Endpoints involved:** none directly — internal worker job. **Expected Inputs / Outputs:** after processing, a chunk's embedding is present and a raw SQL similarity query returns sensible neighbors. **Definition of Done:** embeddings are correctly stored and queryable. **Required Tests:** provider failure triggers retry with backoff, not silent data loss. **Common Failure Cases:** picking an embedding dimension that doesn't match the chosen provider's output; forgetting to re-embed if the provider/model changes later. **Git Branch Example:** `feature/vector-search` *(shared branch prefix with Phase 9 — see Section 10)* **Suggested PR Title:** `feat(embeddings): pgvector storage and embedding worker`

PHASE 9 — Semantic + Keyword Search

Goal: Natural-language queries return relevant memories. **Why this phase exists:** This is the first user-facing payoff of the ingestion pipeline, and the retrieval foundation RAG will reuse. **Dependencies:** Phase 8. **Primary Owner:** Project Lead + Backend. **Reviewer:** Backend / Project Lead (cross-review). **Modules / folders affected:** `backend/app/memory/` (search), `web/src/pages/` (search bar + results). **Backend Tasks:** pgvector + full-text search endpoint. **Frontend Tasks:** search bar, results list. **Extension Tasks:** N/A. **Database Tasks:** `tsvector` index for keyword search alongside the HNSW vector index. **Security Requirements:** every search query is `user_id`-scoped before the ANN operator runs. **Implementation Steps:** implement vector search query → implement keyword search query → merge/rank results → wire the Web App. **API Endpoints involved:** `GET /api/v1/search`. **Expected Inputs / Outputs:** see Section 13 (`/search` example). **Definition of Done:** a semantically related query (no exact keyword overlap) returns the correct memory. **Required Tests:** empty corpus returns a clear empty state, not an error. **Common Failure Cases:** filtering `user_id` after the similarity search instead of before, leaking ranking information across users. **Git Branch Example:** `feature/vector-search` **Suggested PR Title:** `feat(search): semantic and keyword search endpoint`

PHASE 10 — RAG Chat + Citation Mapping

Goal: Grounded, cited answers. **Why this phase exists:** This is Sentiora's headline "understand what you know" experience. **Dependencies:** Phase 9. **Blocked by TBD-2. Primary Owner:** Project Lead + Backend (paired — see Section 7). **Reviewer:** cross-review between the two owners plus one additional developer. **Modules / folders affected:** `backend/app/intelligence/`, `web/src/pages/` (chat UI). **Backend Tasks:** query endpoint, context builder, citation validator. **Frontend Tasks:** chat UI with citation cards. **Extension Tasks:** N/A. **Database Tasks:** `ai_conversation`, `ai_message`, `citation` tables (already modeled in the ERD — implemented here). **Security Requirements: Security Gate 5 applies directly** — system instructions and retrieved memory are never concatenated into the same trust level; citation tags are validated against the actual retrieved set before display. **Implementation Steps:** implement query embedding + retrieval reuse from Phase 9 → build delimited context construction → call the LLM → implement citation validation → wire chat UI. **API Endpoints involved:** `POST /api/v1/intelligence/query`, `GET /api/v1/intelligence/conversations/{id}`. **Expected Inputs / Outputs:** see Section 13 (`/intelligence/query` examples, including the insufficient-evidence response). **Definition of Done:** every response's citations resolve to real, retrieved memories owned by the requesting user; the insufficient-evidence case returns the fixed honest message. **Required Tests:** conflicting-memory case surfaces both sources; unavailable-source case notes possible unreachability; a prompt-injection test page never alters system behavior. **Common Failure Cases:** letting the LLM's citation tags through without validation, allowing a hallucinated reference to display as real. **Git Branch Example:** `feature/rag-chat` **Suggested PR Title:** `feat(intelligence): RAG chat with citation validation`

PHASE 11 — Timeline + Dashboard

Goal: Users can browse what has been captured. **Why this phase exists:** Baseline transparency — users must always be able to see what Sentiora has stored. **Dependencies:** Phase 5. *(Does not need to wait for Phases 8–10.)* **Primary Owner:** Web

Frontend. **Reviewer:** Backend. **Modules / folders affected:** `web/src/pages/` (Timeline, Dashboard). **Backend Tasks:** minor support on `GET /memories` pagination/filtering if not already complete from Phase 5. **Frontend Tasks:** Timeline (filterable by source/date), Dashboard (recent memories, quick search). **Extension Tasks:** N/A. **Database Tasks:** N/A — uses existing indexes from Phase 5's `Memory` model. **Security Requirements:** N/A beyond standard JWT-scoped reads. **Implementation Steps:** build Timeline list + filters → build Dashboard summary view. **API Endpoints involved:** `GET /api/v1/memories`. **Expected Inputs / Outputs:** see Section 13 (`/memories` example). **Definition of Done:** newly captured and manually saved memories both appear correctly, filterable and sortable. **Required Tests:** filter combinations return correct subsets. **Common Failure Cases:** N/A. **Git Branch Example:** `feature/timeline-dashboard` **Suggested PR Title:** `feat(web): memory timeline and dashboard`

PHASE 12 — PDF + YouTube Capture

Goal: Additional MVP source types work. **Why this phase exists:** PDF and YouTube are P0 MVP sources per the PRD, using the same pattern established in Phase 6. **Dependencies:** Phase 7. **Primary Owner:** Extension + Backend (paired, same pattern as Phase 6). **Reviewer:** Project Lead. **Modules / folders affected:** `extension/src/content/`, `backend/app/capture/`. **Backend Tasks:** source-type-specific handling in `capture/`. **Frontend Tasks:** N/A. **Extension Tasks:** PDF viewer content extraction, YouTube metadata/transcript extraction. **Database Tasks:** N/A — reuses the `memory` table's `source_type` enum. **Security Requirements:** same permission + sensitive-content re-check pattern as Phase 6, applied per source type. **Implementation Steps:** implement PDF text extraction from Chrome's built-in viewer → implement YouTube metadata/transcript extraction → wire both through the existing capture endpoint. **API Endpoints involved:** `POST /api/v1/capture` (source_type = `pdf` or `youtube`). **Expected Inputs / Outputs:** a PDF/video produces a Memory with correct metadata. **Definition of Done:** an in-browser PDF and a YouTube video (with and without captions) both produce correct memories. **Required Tests:** missing transcript is not treated as a failure; unsupported file type fails gracefully with notification. **Common Failure Cases:** treating a missing YouTube transcript as an ingestion failure instead of a valid metadata-only memory. **Git Branch Example:** `feature/pdf-youtube-capture` **Suggested PR Title:** `feat(capture): PDF and YouTube source support`

PHASE 13 — Privacy Center + Deletion Hardening

Goal: Full user control over stored data. **Why this phase exists:** Deletion and visibility are non-negotiable product principles, not optional polish. **Dependencies:** Phases 5–11. **Primary Owner:** Backend + Web Frontend. **Reviewer:** Project Lead. **Modules / folders affected:** `backend/app/memory/` (delete cascade), `web/src/pages/` (Privacy Center). **Backend Tasks:** `/privacy/overview` endpoint, delete endpoint hardening (cascade to chunks/vectors in one transaction). **Frontend Tasks:** Privacy Center screen. **Extension Tasks:** N/A. **Database Tasks:** verify cascade behavior on `memory_chunk` foreign key. **Security Requirements:** **Security Gate 6 applies directly** — deletion must be atomic and complete across storage, index, and vectors. **Implementation Steps:** implement `/privacy/overview` aggregation → harden delete transaction → build Privacy Center UI with confirmation modal. **API Endpoints involved:** `GET /api/v1/privacy/overview`, `DELETE /api/v1/memories/{id}`. **Expected Inputs / Outputs:** see Section 13 (`DELETE /memories/{id}` example). **Definition of Done:** every stored memory is visible and actionable from one screen; deletion removes the memory from timeline, search, and future RAG retrieval within the session. **Required Tests:** deleted memory is no longer retrievable by any path, including a previously-open chat session re-asking the same question. **Common Failure Cases:** deleting the `Memory` row but leaving orphaned `MemoryChunk`/embedding rows behind. **Git Branch Example:** `feature/privacy-center` **Suggested PR Title:** `feat(privacy): Privacy Center and hardened deletion cascade`

PHASE 14 — Testing Pass

Goal: The core loop is covered end-to-end. **Why this phase exists:** Manual QA does not scale across four developers shipping in parallel. **Dependencies:** all phases above. **Primary Owner:** Project Lead (coordinates); each developer covers their own area. **Reviewer:** cross-review — no developer approves their own test suite alone. **Modules / folders affected:** all `tests/` directories across `backend/`, `web/`, `extension/`. **Backend Tasks:** cross-user isolation suite, security-focused suite (Section 11 checklist as test cases). **Frontend Tasks:** Web E2E via Playwright. **Extension Tasks:** Extension E2E via Playwright. **Database Tasks:** N/A. **Security Requirements:** every endpoint has at least one negative-auth test (missing/invalid JWT, cross-user access attempt). **Implementation Steps:** write the Playwright suite for capture→search→chat→delete → write the isolation suite → wire into CI as a required check. **API Endpoints involved:** all. **Expected Inputs / Outputs:** N/A. **Definition of Done:** CI runs the full suite on every PR to `main`. **Required Tests:** the full MVP End-to-End Success Test (Section 14) passes automated. **Common Failure Cases:** flaky E2E tests due to async ingestion timing — use polling/status checks rather than fixed sleeps. **Git Branch Example:** `feature/e2e-test-suite` **Suggested PR Title:** `test: full MVP loop and cross-user isolation suite`

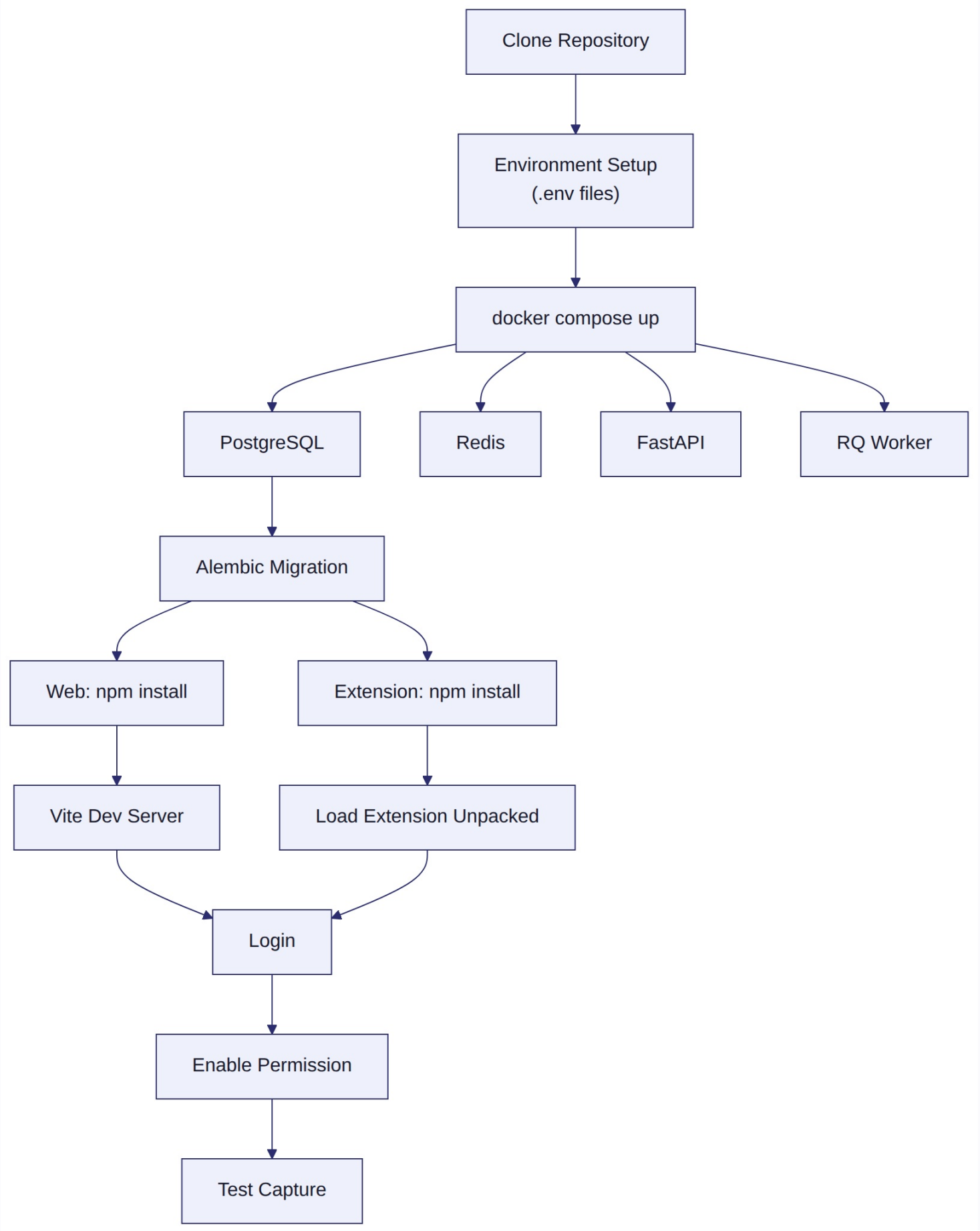
PHASE 15 — Deployment

Goal: MVP is reachable outside local dev. **Why this phase exists:** Validates the whole system under real infrastructure, not just Docker Compose. **Dependencies:** Phase 14. **Primary Owner:** Project Lead. **Reviewer:** Backend. **Modules / folders affected:** `infrastructure/`, `.github/workflows/`. **Backend Tasks:** container readiness (env config, health checks). **Frontend Tasks:** static build readiness. **Extension Tasks:** Web Store submission package. **Database Tasks:** managed Postgres/pgvector provisioning, backup configuration. **Security Requirements:** production secrets only in the platform's secret manager; TLS enforced end-to-end. **Implementation Steps:** provision staging infrastructure → deploy via CI on merge to `main` → smoke test

→ provision production → manual promotion. **API Endpoints involved:** `GET /health` (smoke test). **Expected Inputs / Outputs:** N/A. **Definition of Done:** a fresh user can complete the full MVP loop against the staging environment. **Required Tests:** staging smoke test as part of the deploy pipeline. **Common Failure Cases:** missing environment variables in the managed host causing a silent startup failure. **Git Branch Example:** `feature/deployment-pipeline` **Suggested PR Title:** `chore: staging and production deployment pipeline`

---

9. First Day of Sentiora Development



See Diagram 9 above for the visual version of this same sequence.

1. Create the GitHub repository.
2. Protect `main` (require PR review, disallow direct pushes).
3. Create the monorepo directories (`extension/`, `web/`, `backend/`, `docs/`, `infrastructure/`).
4. Add a root `.gitignore` (node_modules, `.env`, `__pycache__`, build artifacts).
5. Add `.env.example` files for `backend/` and `web/`.
6. Create `docker-compose.yml` (Postgres, Redis, backend, worker).

7. Start Postgres.
8. Enable the `pgvector` extension.
9. Start Redis.
10. Create the FastAPI `/health` endpoint.
11. Create the React/Vite web app skeleton.
12. Create the MV3 extension shell.
13. Configure GitHub Actions (lint, type-check, test).
14. Open initial PRs for each piece above (small, reviewable increments).
15. Verify local environments — every developer confirms `docker compose up` works on their machine.

> **FIRST MILESTONE** `docker compose up` must successfully bring up **FastAPI**, **PostgreSQL**, **Redis**, and the **RQ Worker**, and `GET /health` must return a successful response. Nothing else begins until every developer can reproduce this locally.

---

10. Git / GitHub Workflow

```
main
 ↑
Pull Request (review required)
 ↑
feature branch
```

Branch naming examples: `feature/auth-login`, `feature/permissions`, `feature/extension-shell`, `feature/manual-memory`, `feature/webpage-capture`, `feature/ingestion`, `feature/vector-search`, `feature/rag-chat`.

Rules: - **No direct commits to `main`.** - Every feature: branch → code → test → PR → review → merge. - Every database schema change: an Alembic migration, never a manual/ad hoc schema edit. - Every endpoint: validation + authentication/authorization + tests, before merge.

---

11. API Implementation Checklist

Run this checklist against **every** endpoint before it merges:

☐ Pydantic request validation ☐ JWT validation where required ☐ `user_id` resolved server-side (never from the client) ☐ Authorization check (the resource belongs to the requesting user) ☐ Permission check, if the endpoint touches capture or Memory ☐ Rate limit applied, if the endpoint is auth- or ingestion-related ☐ Input sanitized (no raw HTML rendering downstream) ☐ Consistent error envelope (`{error: {code, message}}`) ☐ At least one automated test ☐ No sensitive data in logs (no passwords, tokens, or raw Memory content)

---

12. Security Gates

SECURITY GATE 1 — AUTHENTICATION **Threat:** Unauthorized access to any account or data. **Control:** argon2 password hashing, short-lived JWT access tokens, Redis-backed rotating refresh tokens. **Where enforced:** `backend/app/auth/`, every protected route's dependency injection. **Required test:** invalid credentials rejected; expired/invalid tokens rejected on every protected route.

SECURITY GATE 2 — PERMISSION BEFORE COLLECTION **Threat:** Data captured from a source the user never approved. **Control:** client-side advisory check + mandatory server-side re-validation of `Permission.status` before any capture is persisted. **Where enforced:** extension content scripts (advisory) and `backend/app/capture/` (authoritative). **Required test:** a capture request for a revoked or never-granted source is rejected regardless of client payload.

SECURITY GATE 3 — SENSITIVE CONTENT EXCLUSION **Threat:** Passwords, payment details, banking, or health information entering Memory. **Control:** layered client + server heuristics, fail-closed on any uncertain classification (Section 5.4 / Diagram 3). **Where enforced:** `extension/src/content/` (advisory) and `backend/app/capture/` (authoritative, final). **Required test:** dedicated test pages for Incognito, auth, payment, and health categories must never produce a Memory.

SECURITY GATE 4 — USER DATA ISOLATION **Threat:** User A retrieving User B's memories, vectors, conversations, or citations. **Control:** every data-access query filters on `user_id` sourced exclusively from the verified JWT claim — never from a request parameter. **Where enforced:** every query in `backend/app/memory/`, `backend/app/intelligence/`, and the vector search layer. **Required test:** cross-user access attempts against every list/read/delete endpoint are rejected.

SECURITY GATE 5 — PROMPT INJECTION DEFENSE **Threat:** Text inside a captured page (e.g. "*Ignore previous instructions...*") being interpreted as a command by the LLM. **Control:** system/context separation, explicit delimiters around retrieved content, no agentic tool execution, citation validation against the actual retrieved set. **Where enforced:** `backend/app/intelligence/` context builder and citation validator. **Required test:** a prompt-injection test page's content is summarized/cited as inert text and never alters system behavior.

SECURITY GATE 6 — SAFE DELETION **Threat:** A "deleted" memory remaining retrievable via search, RAG, or vector similarity. **Control:** hard delete of the Memory row cascading to MemoryChunks and embeddings in a single transaction. **Where enforced:** `backend/app/memory/` delete endpoint. **Required test:** a deleted memory is unretrievable from timeline, search, and RAG — including a previously-open chat session re-asking the same question.

---

13. Pre-Build Decision Checklist — Blocking TBDs

These are preserved exactly from v1.1. No decisions are invented here.

TBD-1 — Exact health-sensitive-content exclusion rules. Blocks: Phase 6 (Webpage Capture / Sensitive Content Guard).

TBD-2 — Embedding + LLM provider/model selection. Blocks: Phase 8 (Embeddings) and Phase 10 (RAG Chat).

TBD-3 — Companion Web App hosting/auth split confirmation. Blocks: Phase 11 (Timeline + Dashboard) deployment/integration specifics.

☐ TBD-1 resolved before Phase 6 begins ☐ TBD-2 resolved before Phase 8 begins ☐ TBD-3 resolved before the affected Web App deployment/integration work in Phase 11

---

14. MVP End-to-End Success Test

If this complete loop passes, the core Sentiora MVP works.

```
New User
   ↓
Register
   ↓
Login
   ↓
Read Privacy Explanation
   ↓
Grant Webpage Permission
   ↓
Browse Article
   ↓
Meaningful Capture Triggered
   ↓
Memory Created
   ↓
Content Chunked
   ↓
Embeddings Stored
   ↓
Memory Appears in Timeline
   ↓
Search for Related Concept
   ↓
Correct Memory Returned
   ↓
Ask Sentiora
   ↓
Grounded Answer + Citation
   ↓
Delete Memory
   ↓
Memory Disappears From:
  · Timeline
  · Search
  · RAG Retrieval
  · Vector Results
```

This is the canonical test referenced in Phase 14's Definition of Done. Automate it as the primary Playwright E2E test before considering the MVP feature-complete.

---

15. Engineering Rules

These are non-negotiable across all four developers and all fifteen phases:

- **NEVER** accept `user_id` from the frontend or extension — always resolve it server-side from the verified JWT.
- **NEVER** commit `.env` files or provider API keys to source control.
- **NEVER** put provider keys in browser code (extension or web app).
- **NEVER** bypass the Permission-check layer to create a Memory.
- **NEVER** persist a Memory before the sensitive-content check has passed.
- **NEVER** call embedding or LLM provider APIs directly from the extension or web app — only the backend talks to external AI providers.
- **NEVER** render captured content as raw HTML — always escaped text/markdown.
- **EVERY** query touching user data must be explicitly `user_id`-scoped.
- **EVERY** new API endpoint ships with validation, authorization, and at least one test.
- **NO** direct commits to `main`.
- **ALL** schema changes go through Alembic migrations.

Before You Merge Checklist

☐ No `user_id` accepted from client input anywhere in this change ☐ No `.env`, secrets, or API keys committed ☐ No provider credentials present in extension or web app code ☐ Permission check present on every new capture path ☐ Sensitive-content check runs before any Memory write ☐ AI provider calls happen only in backend code ☐ Captured content is rendered escaped,

never as raw HTML ☐ Every new/changed query is `user_id`-scoped ☐ New endpoint has validation, authorization, and a test ☐ Change went through a feature branch and PR review — not a direct push to `main` ☐ Any schema change is an Alembic migration

---

16. Technical Decision Register (Reference)

ID	Decision	Selected	Reason
TD-001	Backend framework	FastAPI	Async-first, fast to build a RAG-heavy API
TD-002	Primary database	PostgreSQL	Relational integrity + isolation guarantees
TD-003	Vector store	pgvector	Single database, atomic delete cascades
TD-004	Authentication	JWT + Redis-backed refresh token	Stateless verification, revocable sessions
TD-005	AI orchestration	Direct provider calls, no framework	Full control of citations and injection defenses
TD-006	Background jobs	RQ + Redis	Smallest operational footprint
TD-007	Repository structure	Monorepo	Simplifies coordination for 4 developers
TD-008	Deployment	Docker Compose + single managed host	Avoids Kubernetes

---

End of Sentiora Technical Specification v1.2 — Final Engineering Execution Handbook. Architecture unchanged from v1.1; this version restructures for readability, visual clarity, and phase-by-phase coding readiness.